

Approach

I implemented a lightweight classical computer vision pipeline rather than training a deep neural network. The detector is based on the observation that photographs of digital displays contain subtle forensic artifacts that are generally absent in genuine camera photographs. Instead of recognizing scene content, the model analyzes image characteristics such as frequency-domain patterns, texture, noise statistics, color consistency, JPEG artifacts, edge sharpness, chromaticity, and specular reflections.

For each input image, a 68-dimensional handcrafted feature vector is extracted. These features are normalized using a StandardScaler and passed to a soft voting ensemble consisting of an RBF Support Vector Machine, Logistic Regression, and Random Forest classifier. The ensemble outputs a probability between 0 and 1, where values closer to 1 indicate a higher likelihood that the image is a photograph of a digital screen. The implementation also includes a classical feature-based fallback mode if a trained model is unavailable.

Training Data

The model was trained using a combination of two public academic datasets. The ICVIP 2020 Recaptured Image Forensics dataset provides genuine camera photographs and corresponding recaptured images photographed from digital displays. To increase diversity, I also incorporated 500 original images and 500 corresponding recaptured (moire) images from the UHDM (ECCV 2022) dataset. After augmentation, the effective training set contained approximately 12,852 samples.

Accuracy

Five-fold stratified cross-validation on the training dataset achieved approximately **96.3% $\pm$ 0.8% accuracy**. This is the honest cross-validation performance reported by the training pipeline and should not be interpreted as the expected performance on unseen deployment data. Performance on unseen data ultimately depends on the distribution of the evaluation images.

Manual Validation

In addition to cross-validation, I manually tested the detector on approximately 25–30 images captured under different lighting conditions, viewing angles, devices, and screen brightness levels. These images were used only for validation, not training, helping identify practical failure cases and improve robustness.

Latency

Measured on a laptop CPU, inference typically takes **approximately 50 to 70 ms per image**, including feature extraction and model prediction, making the detector suitable for real-time or near real-time use on CPU hardware.

Cost per Image

The intended deployment is entirely **on-device**, so the cost per image is effectively **\$0**, with no server infrastructure or network communication required. If deployed on a cloud server, the computational cost would remain very low because inference consists only of handcrafted feature extraction followed by a small classical machine learning ensemble.

Future Improvements

Given more time, I would improve robustness for two challenging scenarios. The first is extremely low-brightness screens, where display artifacts become much weaker. The second is printed photographs, which exhibit different forensic characteristics than digital displays and would benefit from dedicated print-specific features and additional training data. I would also continue collecting more diverse real-world examples from different devices, display technologies, lighting conditions, and camera models to further improve generalization on unseen data.